In C++, inheritance allows one class (the derived class) to inherit properties and behaviors (members and methods) from another class (the base class). There are several types of inheritance in C++:

1. Single Inheritance:

In single inheritance, a derived class inherits from a single base class.

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived : public Base {
public:
    void display() {
        std::cout << "Derived class" << std::endl;
    }
};

int main() {
    Derived obj;
    obj.show();     // Base class method
    obj.display();  // Derived class method
    return 0;
}
```

2. Multiple Inheritance:

In multiple inheritance, a derived class inherits from more than one base class.

Example:

```
class Base1 {
public:
    void show() {
        std::cout << "Base1 class" << std::endl;
    }
};

class Base2 {
public:
    void display() {
        std::cout << "Base2 class" << std::endl;
    }
};

class Derived : public Base1, public Base2 {
public:
    void showMessage() {
        std::cout << "Derived class" << std::endl;
    }
};
```

```
int main() {
    Derived obj;
    obj.show();      // Base1 class method
    obj.display();   // Base2 class method
    obj.showMessage();  // Derived class method
    return 0;
}
```

3. Multilevel Inheritance:

In multilevel inheritance, a derived class is derived from another derived class (i.e., there is a chain of inheritance).

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : public Base {
public:
    void display() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : public Derived1 {
public:
    void showMessage() {
        std::cout << "Derived2 class" << std::endl;
    }
};

int main() {
    Derived2 obj;
    obj.show();      // Base class method
    obj.display();   // Derived1 class method
    obj.showMessage(); // Derived2 class method
    return 0;
}
```

4. Hierarchical Inheritance:

In hierarchical inheritance, multiple derived classes inherit from a single base class.

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};
```

```
class Derived1 : public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : public Base {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

int main() {
    Derived1 obj1;
    Derived2 obj2;
    obj1.show();
    obj1.display1();
    obj2.show();
    obj2.display2();
    return 0;
}
```

5. Hybrid Inheritance:

Hybrid inheritance is a combination of two or more types of inheritance. For example, combining multiple and multilevel inheritance.

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

class FinalDerived : public Derived1, public Derived2 {
public:
    void showMessage() {
        std::cout << "FinalDerived class" << std::endl;
    }
};
```

```
int main() {
    FinalDerived obj;
    obj.show();          // Base class method
    obj.display1();      // Derived1 class method
    obj.display2();      // Derived2 class method
    obj.showMessage();   // FinalDerived class method
    return 0;
}
```

6. Virtual Inheritance:

Virtual inheritance is used to solve the problem of *diamond problem* in multiple inheritance. When two or more base classes share a common ancestor, virtual inheritance ensures that the shared base class is only inherited once.

Example:

```
class Base {
public:
    void show() {
        std::cout << "Base class" << std::endl;
    }
};

class Derived1 : virtual public Base {
public:
    void display1() {
        std::cout << "Derived1 class" << std::endl;
    }
};

class Derived2 : virtual public Base {
public:
    void display2() {
        std::cout << "Derived2 class" << std::endl;
    }
};

class FinalDerived : public Derived1, public Derived2 {
public:
    void showMessage() {
        std::cout << "FinalDerived class" << std::endl;
    }
};

int main() {
    FinalDerived obj;
    obj.show();          // Base class method (only one call due to virtual
inheritance)
    obj.display1();      // Derived1 class method
    obj.display2();      // Derived2 class method
    obj.showMessage();   // FinalDerived class method
    return 0;
}
```

Summary:

- **Single Inheritance**: A derived class inherits from one base class.
- **Multiple Inheritance**: A derived class inherits from more than one base class.
- **Multilevel Inheritance**: A derived class inherits from another derived class.
- **Hierarchical Inheritance**: Multiple derived classes inherit from a single base class.
- **Hybrid Inheritance**: A combination of two or more inheritance types.
- **Virtual Inheritance**: Used to handle the diamond problem and ensure a base class is inherited only once.

Each type of inheritance in C++ allows you to structure your classes differently depending on the relationships between the entities.